

Guía de estudio e implementación

Automatización del restaurante del Colo

Reservas y pedidos por WhatsApp con IA

11 días de estudio, del 4 al 14 de agosto de 2026

Accelerate.ai — Nacho & Colo

Contexto del proyecto

QUIÉNES SOMOS

Accelerate.ai es la agencia de automatizaciones con inteligencia artificial que están armando Nacho y Colo. Nacho aporta programación y automatización; Colo aporta Excel, Power BI y administración — y cada uno está aprendiendo también del área del otro, para tener los dos una visión general de la agencia frente a un cliente.

QUÉ ES ESTE DOCUMENTO

Es la guía técnica paso a paso para construir el primer caso real de la agencia: una automatización de WhatsApp con inteligencia artificial para el restaurante donde trabaja Colo. Cada día combina la teoría con la construcción real, para llegar al 14 de agosto con algo funcionando, no solo estudiado.

Una aclaración importante: todo el código de estos 11 días construye específicamente una herramienta llamada crear_reserva, pensada para tomar reservas de mesa. Ese es el ejemplo elegido para desarrollar mientras se espera la confirmación de Colo sobre si el foco final va a ser reservas, pedidos, o ambos - no es una decisión cerrada. Si termina siendo pedidos (o los dos), el patrón completo (tool use, RAG, persistencia en Sheets) es exactamente el mismo; lo único que cambia son los campos de la tool y el texto de los mensajes.

ARQUITECTURA COMPLETA

Así se conectan todas las piezas que se van a ir construyendo, día por día:



El cliente escribe por WhatsApp; n8n conecta todo con el servidor propio (FastAPI), que a su vez habla con Claude para entender el pedido, con Voyage para buscar en el FAQ, y con Google Sheets para dejar la reserva guardada.

Contenido

Contexto del proyecto	2
Antes de empezar: preparación del entorno	4
Cómo vamos a trabajar: guía + Project + Claude Code	6
Sección 1 — El caño completo	7
Día 1 — 4 de agosto	7
Día 2 — 5 de agosto	8
Sección 2 — Claude como agente	10
Día 3 — 6 de agosto	10
Día 4 — 7 de agosto	13
Día 5 — 8 de agosto	14
Sección 3 — RAG casero para el menú y las FAQ	16
Día 6 — 9 de agosto	16
Día 7 — 10 de agosto	17
Día 8 — 11 de agosto	19
Sección 4 — Pulido y testing de punta a punta	20
Día 9 — 12 de agosto	20
Día 10 — 13 de agosto	21
Día 11 — 14 de agosto	22
Glosario	24

Antes de empezar: preparación del entorno

Antes de escribir la primera línea del día 1, conviene tener listas las cuentas y herramientas de base — para no perder tiempo de estudio real resolviendo trámites a mitad de camino.

CUENTAS Y ACCESOS NECESARIOS

No todos los trámites de cuentas son igual de urgentes. Separarlos entre lo que bloquea el arranque y lo que se puede resolver un poco más adelante evita perder el primer día entero en formularios.

Imprescindible antes de arrancar el día 1 (sin esto no se puede ni empezar)

- Python 3.11 o superior instalado en la máquina.
- Cuenta gratuita en ngrok.com, con el authtoken configurado localmente — se necesita recién el día 2, pero como no tiene costo ni espera, conviene resolverla ya de una.
- Cuenta de Meta for Developers (developers.facebook.com), con una app creada y el producto WhatsApp agregado — esto habilita el número de prueba gratuito. Este trámite es instantáneo mientras se use el número de prueba: la verificación de negocio de varios días hábiles recién hace falta el día que se quiera pasar al número real del restaurante, algo que queda fuera de estos 11 días.

Se puede postergar unos días (necesario recién a partir de los días indicados)

- Cuenta en console.anthropic.com, con una API key generada y créditos cargados — necesaria para el día 3, no antes. Los días 1 y 2 no usan la API de Claude todavía.
- Cuenta de Google lista para usar Sheets desde n8n — necesaria para el día 4.
- Cuenta en voyageai.com, con su propia API key — necesaria para el día 6, cuando arranca RAG.

En resumen: para hoy y mañana (días 1 y 2) alcanza con Python, ngrok y Meta. El resto se puede ir resolviendo en paralelo durante esos mismos dos días, sin que bloquee nada todavía.

INSTALACIÓN LOCAL

Python 3.11 o superior instalado, y un entorno virtual propio para este proyecto, para no mezclar dependencias con otros proyectos que ya tengas en la máquina.

```
python -m venv venv
source venv/bin/activate # en Windows: venv\Scripts\activate

pip install fastapi uvicorn pydantic anthropic voyageai numpy python-dotenv requests
```

Ese mismo listado de paquetes conviene dejarlo en un archivo requirements.txt en la raíz del proyecto, para poder reinstalar todo con un solo comando si hace falta:

```
fastapi
uvicorn
pydantic
anthropic
voyageai
numpy
python-dotenv
requests
```

VARIABLES DE ENTORNO

Las claves de API nunca van escritas directo en el código. Se guardan en un archivo `.env` en la raíz del proyecto (que nunca se sube a git), y se cargan desde ahí con `python-dotenv`:

```
# archivo .env
ANTHROPIC_API_KEY=sk-ant-...
VOYAGE_API_KEY=pa-...
WHATSAPP_TOKEN=...
WHATSAPP_PHONE_ID=...
```

```
from dotenv import load_dotenv
import os

load_dotenv()
ANTHROPIC_API_KEY = os.environ["ANTHROPIC_API_KEY"]
```

Antes del primer commit, conviene inicializar el repo de git para este proyecto y agregar un `.gitignore` que excluya el archivo `.env` y la carpeta `venv/` — así nunca se sube una clave por accidente. Cerrar cada día de estudio con un commit al repo (el mismo hábito que ya venís usando en `estudio-ai-2026`, o uno nuevo para este proyecto puntual) deja un historial real de avance, y sirve como respaldo si algo se rompe.

UNA NOTA BREVE DE COSTOS

Usar la API de Claude y la de Voyage tiene costo por uso, aunque en el volumen de pruebas de estos 11 días es mínimo (el orden de centavos de dólar en total). La API de WhatsApp Cloud tiene un nivel gratuito para desarrollo con el número de prueba. Ninguno de estos gastos debería impactar el presupuesto mensual ya pensado para herramientas.

Cómo vamos a trabajar: guía + Project + Claude Code

Este proyecto se va a apoyar en tres piezas distintas, cada una con un rol propio que no se superpone con las otras — vale la pena tenerlo claro desde el principio para no mezclarlas.

LA GUÍA (ESTE PDF)

Es la referencia de fondo: la explicación en profundidad de cada concepto, la parte pensada para que Colo entienda qué está pasando, y el paso a paso de qué se construye cada día. Se lee primero, antes de tocar código.

EL PROJECT DE CLAUDE (ACÁ, EN LOS CHATS)

Se va a crear un proyecto acá en Claude con este PDF cargado como contexto, para que cualquier chat nuevo dentro de ese proyecto ya sepa de qué se trata todo, sin tener que reexplicarlo cada vez. Ese espacio sirve para tres cosas puntuales: entender un concepto del día que no haya quedado claro solo con la lectura, resolver dudas específicas antes de escribir código, y decidir qué le vamos a pedir a Claude Code que haga en cada paso — qué escribe, qué corre, qué debuggea. Es la instancia de pensar y decidir, no la de escribir código.

CLAUDE CODE (EN LA CARPETA DEL PROYECTO, EN VS CODE)

Es quien efectivamente escribe, corre y debuggea el código real, dentro de la carpeta del proyecto. La idea es que, a medida que va escribiendo cada parte, vaya explicando técnicamente qué está haciendo y por qué — nadie mejor para explicar el código que quien lo está escribiendo en ese momento. El setup de Claude Code para este proyecto (el archivo CLAUDE.md y el context.md) se arma de cero, sin reutilizar el que ya existe del intento anterior de Accelerate.ai en solitario: este es un proyecto distinto, con Colo como socio, y conviene que su configuración refleje eso desde el principio.

EL FLUJO SUGERIDO, DÍA POR DÍA

- 1. Leer el día completo en esta guía — el concepto en profundidad y la parte para Colo.
- 2. Si algo no quedó claro, resolverlo en el Project acá en los chats antes de tocar código.
- 3. Recién ahí, abrir Claude Code y pedirle que ayude a construir la parte de “qué construimos hoy”, pidiéndole explícitamente que explique cada paso a medida que lo escribe.
- 4. En los conceptos genuinamente nuevos del día, escribir el código a mano una primera vez, aunque sea más lento — reservar Claude Code para lo repetitivo, el boilerplate, y el debugging cuando algo falla.

Esta combinación es justamente la que evita el riesgo de terminar con el bot funcionando pero sin la habilidad real adentro: el Project obliga a pensar antes de escribir, y pedirle a Claude Code que explique mientras construye convierte cada día en una segunda explicación técnica, no solo en código copiado.

Sección 1 — El caño completo

Día 1 — 4 de agosto

Levantar el primer endpoint en FastAPI que reciba un mensaje y devuelva una respuesta, usando Pydantic para validar la forma del dato.

EL CONCEPTO, EN PROFUNDIDAD

FastAPI es un framework para construir APIs web en Python, construido sobre dos piezas: Starlette (que maneja el ruteo, las peticiones HTTP y los websockets a bajo nivel) y Pydantic (que valida los datos usando las anotaciones de tipo que ya escribis en Python normal). La unidad básica es la "path operation": una función decorada con `@app.get(...)`, `@app.post(...)`, etc., asociada a una URL y un metodo HTTP. Cuando declaras los parametros de esa función con tipos (str, int, o un modelo de Pydantic), FastAPI hace tres cosas automáticamente: valida que el dato que llega tenga esa forma, lo convierte al tipo de Python correspondiente, y genera documentación interactiva (Swagger UI en /docs) sin que escribas nada aparte.

Un modelo de Pydantic es una clase que hereda de BaseModel, donde cada atributo declarado es un campo con su tipo. Si el dato que llega no cumple ese tipo, Pydantic devuelve un error 422 automáticamente, antes de que tu código llegue siquiera a ejecutarse - esto es lo que te ahorra escribir validaciones manuales en cada endpoint.

La otra decisión importante del día 1 es `async def` contra `def` normal. Mientras una función `async` esta esperando algo lento (una llamada de red, por ejemplo), el proceso puede atender otras peticiones en simultáneo, en vez de quedarse bloqueado. Para un webhook que más adelante va a llamar a Claude y a WhatsApp (dos llamadas de red), declarar el endpoint como `async def` desde el día uno evita tener que reescribirlo después.

Por último, Uvicorn es el servidor que efectivamente corre tu aplicación: FastAPI define QUE hace tu código, Uvicorn es el programa que lo pone a escuchar peticiones de verdad en un puerto de tu máquina.

Aplicado a Accelerate.ai: cada automatización que vendamos de acá en adelante -no solo la de este restaurante- va a necesitar este mismo tipo de servidor recibiendo datos y devolviendo respuestas. Lo que hoy parece un ejercicio chico (un endpoint que hace eco) es, en los hechos, la plantilla técnica que se reutiliza para el bot productizado (idea 2 del ranking) y para los dashboards (idea 3): todos arrancan de la misma base de FastAPI más Pydantic. Aprenderla bien hoy acorta el tiempo de armar cada cliente futuro.

PARA COLO

Hoy armamos la base técnica de todo el proyecto: un servidor (un programa que queda escuchando peticiones por internet) que recibe un mensaje y devuelve una respuesta. Ese mecanismo estándar por el que dos sistemas distintos se pasan información — en este caso, WhatsApp y nuestro programa — es lo que se conoce como una API. Hoy todavía no tiene ninguna inteligencia: si le llega "hola", devuelve "hola". Pero es la pieza sobre la que se monta absolutamente todo lo que viene después, así que conviene que quede sólida desde el primer día.

DOCUMENTACIÓN OFICIAL DEL DÍA

- FastAPI, tutorial oficial: <https://fastapi.tiangolo.com/tutorial/>
- Pydantic v2, guía de modelos: <https://docs.pydantic.dev/latest/>

QUÉ CONSTRUIMOS HOY

Un archivo main.py con un endpoint POST /webhook, un modelo Pydantic MensajeEntrante (con campos número y texto), y la app corriendo local con uvicorn main:app --reload.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class MensajeEntrante(BaseModel):
    numero: str
    texto: str

@app.post("/webhook")
async def recibir_mensaje(mensaje: MensajeEntrante):
    # por ahora, devolvemos un eco
    return {"respuesta": f"Recibido: {mensaje.texto}"}
```

- **from fastapi import FastAPI, from pydantic import BaseModel:** importamos las dos piezas de la teoría de hoy - FastAPI para el servidor, BaseModel como base para definir la forma de los datos.
- **app = FastAPI():** crea la aplicación. Es el objeto central al que se le van a ir agregando todos los endpoints de acá en adelante, no solo el de hoy.
- **class MensajeEntrante(BaseModel), con número: str y texto: str:** le decimos a Pydantic exactamente que forma tiene que tener el mensaje que llega. Si falta alguno de los dos campos, o vienen con el tipo equivocado, FastAPI rechaza la petición automáticamente antes de que nuestro código la vea.
- **@app.post("/webhook"):** registra la URL /webhook para que responda a peticiones POST - el mismo path que después va a usar n8n para mandarnos los mensajes reales.
- **async def recibir_mensaje(mensaje: MensajeEntrante):** declarar el parametro con el tipo MensajeEntrante es lo que dispara toda la validación de Pydantic automáticamente, sin que escribamos ningún chequeo manual.
- **El return con el f-string:** por ahora es un eco simple, pero es el mismo lugar exacto donde el día 4 va a ir la confirmación real de la reserva.
- **uvicorn main:app --reload:** main es el nombre del archivo, app es el objeto que creamos arriba, y --reload hace que el servidor se reinicie solo cada vez que guardamos un cambio.

RESULTADO ESPERADO AL CERRAR EL DÍA

Correr uvicorn main:app --reload y, desde Postman o curl, mandar un POST a <http://127.0.0.1:8000/webhook> con {"numero": "123", "texto": "hola"} y recibir {"respuesta": "Recibido: hola"}.

Día 2 — 5 de agosto

Exponer el endpoint a internet con ngrok, conectar un número de WhatsApp de prueba de Meta, y armar el flujo mínimo en n8n que una todo.

EL CONCEPTO, EN PROFUNDIDAD

Tu servidor corre en 127.0.0.1, una dirección que solo existe dentro de tu propia máquina - nada de afuera puede llegar ahí. ngrok resuelve esto abriendo un túnel: le decís que exponga el puerto 8000, y te da una URL pública que reenvía cualquier petición que le llegue directo a tu localhost:8000. Es una solución de desarrollo - cuando el restaurante use esto en producción va a vivir en un servidor real, pero para probar mientras se construye, ngrok es exactamente lo que se usa.

Meta ofrece, dentro de cada app de desarrollador, un número de teléfono de prueba gratuito, sin necesitar la verificación de negocio completa (esa verificación de días hábiles es solo para pasar a un número de producción real, más adelante). Con ese número alcanza para desarrollar y probar todo el flujo.

Cuando registras la URL de tu webhook en el panel de Meta, antes de empezar a mandarte mensajes reales, Meta hace una verificación: manda una petición GET con un parámetro `hub.challenge`, y tu sistema tiene que devolver exactamente ese valor para confirmar que el webhook es tuyo. Recién después, Meta empieza a mandar los mensajes entrantes como peticiones POST.

Acá es donde entra n8n, y por que conviene meterlo ya en el medio en vez de que Meta le hable directo a tu FastAPI: n8n tiene un nodo WhatsApp Trigger que ya sabe manejar esa verificación de Meta y recibir los mensajes entrantes sin que escribas ese código vos. Desde ahí, un nodo HTTP Request reenvía el mensaje a tu endpoint de FastAPI (la URL de ngrok), espera la respuesta, y un segundo HTTP Request la manda de vuelta a la API de WhatsApp. La ventaja de este layout: n8n te da un registro visual de cada mensaje que pasa por ahí, y más adelante te deja sumar pasos (guardar en Sheets, mandar una alerta) sin tocar el código Python.

Dos limitaciones reales de WhatsApp que conviene conocer desde ahora, aunque no las resolvamos hoy: primero, Meta solo deja mandar mensajes de texto libre dentro de una ventana de 24 horas desde el último mensaje del cliente - para mandar algo por fuera de esa ventana (un recordatorio proactivo, por ejemplo) hace falta un mensaje de plantilla pre-aprobado por Meta, no texto libre. Segundo, mientras uses el número de prueba, solo le podés escribir a números que hayas agregado a mano en una lista de destinatarios permitidos dentro del panel de Meta - si te olvidas de agregar tu propio número ahí, el bot va a parecer roto cuando en realidad es un paso de configuración que falta.

Aplicado a Accelerate.ai: este circuito (WhatsApp real conectado a nuestro código a través de n8n) es exactamente lo que hay que repetir con cada cliente nuevo de la agencia. Una vez que lo entendemos bien hoy, conectar el número del segundo, tercer o cuarto cliente es repetir este mismo procedimiento con datos distintos, no aprenderlo de cero - es la parte que se vuelve plantilla reutilizable de la idea 1 del ranking (agencia general de automatizaciones).

PARA COLO

Hoy conectamos ese servidor a un número de WhatsApp real, aunque todavía de prueba, no el del restaurante. El camino que recorre un mensaje es: tu WhatsApp, los servidores de Meta (la empresa dueña de WhatsApp), una herramienta llamada n8n que actúa de intermediario y reenvía el mensaje a nuestro servidor, y de ahí la respuesta vuelve por el mismo camino. Meta exige un paso de verificación técnica antes de dejarnos recibir mensajes reales, así que parte de hoy es superar ese trámite. Al final del día, el circuito completo va a estar armado y probado con un mensaje real, aunque el contenido de la respuesta siga siendo mínimo.

DOCUMENTACIÓN OFICIAL DEL DÍA

- ngrok, getting started: <https://ngrok.com/docs/start>
- Meta, WhatsApp Cloud API - Get Started: <https://developers.facebook.com/docs/whatsapp/cloud-api/get-started/>
- n8n, nodo WhatsApp Trigger: <https://docs.n8n.io/integrations/builtin/trigger-nodes/n8n-nodes-base.whatsapptrigger>
- n8n, nodo Webhook: <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook>

QUÉ CONSTRUIMOS HOY

ngrok corriendo sobre el servidor del día 1; una app de desarrollador en Meta con el número de prueba activado; un workflow en n8n con WhatsApp Trigger, seguido de un HTTP Request a tu URL de ngrok, seguido de un HTTP Request de vuelta a la API de WhatsApp con la respuesta.

- **ngrok http 8000:** expone tu puerto 8000 (donde corre Uvicorn) con una URL pública tipo <https://xxxx.ngrok-free.app> - esa URL es la que le vamos a dar a Meta y a n8n. Ojo: en el plan gratuito, esa URL cambia cada vez que reinicias ngrok, así que conviene dejarlo corriendo sin cerrarlo durante estos 11 días en vez de reiniciarlo a cada rato - si cambia, hay que actualizarla tanto en n8n como en el panel de Meta.
- **App de desarrollador en Meta, con WhatsApp agregado como producto:** el paso administrativo que habilita el número de prueba y da el token de acceso temporal, el ID de la cuenta de WhatsApp Business, y el ID del número de teléfono - los tres se ven en la app, sección WhatsApp > Primeros pasos.
- **Lista de destinatarios permitidos:** en esa misma sección hay que agregar tu número de celular (y el de Colo) como destinatario de prueba, o los mensajes del bot no van a llegar a ningún lado.
- **Nodo WhatsApp Trigger en n8n:** se configura con el token, el ID de cuenta y el ID del número. Además pide un verify token: un texto que vos inventás, y que tiene que ser exactamente el mismo que cargues en el panel de Meta al registrar la URL del webhook - internamente el nodo ya sabe responder al challenge de verificación de Meta, así que no hay que programar esa parte a mano.
- **Nodo HTTP Request (1):** método POST, URL = tu URL de ngrok + /webhook, Body Content Type = JSON, con el body armado a partir de los campos número y texto que vienen en el mensaje entrante de WhatsApp Trigger - conecta n8n con tu FastAPI.
- **Nodo HTTP Request (2):** método POST, URL = https://graph.facebook.com/v21.0/{ID_NUMERO}/messages, con un header Authorization: Bearer {token}, y el body en el formato que exige la API de WhatsApp (messaging_product, to, type, text) - toma la respuesta que devolvió tu FastAPI y se la manda al cliente real.

RESULTADO ESPERADO AL CERRAR EL DÍA

Mandar "hola" desde tu WhatsApp personal al número de prueba, y recibir "Recibido: hola" de vuelta, por el canal real.

Sección 2 — Claude como agente

Día 3 — 6 de agosto

Definir la herramienta (tool) que le permite a Claude extraer los datos de una reserva o pedido a partir de un mensaje en lenguaje libre.

EL CONCEPTO, EN PROFUNDIDAD

Tool use (function calling) es la forma en que Claude interactúa con funciones que vos definís. Le mandas, junto con el mensaje del usuario, una lista de "herramientas" disponibles - cada una descrita como un objeto con tres partes: name (el nombre de la función), description (una explicación en lenguaje natural de que hace y cuando

usarla - esta es la parte que más peso tiene, porque el modelo decide si usar la herramienta basandose en gran parte en esta descripción) e `input_schema` (un JSON Schema que define que parametros necesita, con sus tipos, y cuales son obligatorios via `required`).

Es fundamental entender que Claude no ejecuta la función - solo decide que hay que usarla y devuelve, en su respuesta, un bloque de tipo `tool_use` con el nombre de la herramienta y los argumentos ya completados según lo que entendio del mensaje. La ejecución real (guardar la reserva, consultar disponibilidad) la hace tu código, no el modelo. Esa distinción es la que separa un "agente" de un simple generador de texto: el modelo decide que acción corresponde y con que datos, tu sistema la lleva a cabo.

Para una reserva de restaurante, la herramienta necesita campos como fecha, hora, cantidad_personas y nombre, marcando como `required` los que no pueden faltar para que la reserva sea valida.

Aplicado a Accelerate.ai: el patron de definir una tool con `name`, `description` e `input_schema` es el mismo que se usa para cualquier automatización futura de la agencia, no solo para reservas de restaurante: un bot de turnos para una peluqueria, un bot de pedidos para otro rubro, todos arrancan definiendo que información necesita extraer la IA. Dominar bien este día acorta muchísimo el tiempo de armar el proximo cliente.

PARA COLO

Hoy le damos al modelo de IA (Claude) una definición formal de qué información necesita para registrar una reserva: fecha, hora, cantidad de personas, nombre. Esto se llama `function calling` o `tool use` — en vez de que el modelo devuelva texto libre que después tendríamos que interpretar a mano, le pedimos que entregue los datos ya organizados en un formato fijo, como si completara un formulario. El modelo decide cuándo corresponde usar ese formulario según lo que dice el cliente, pero no ejecuta ninguna acción por sí mismo: solo entrega los datos, y nuestro código decide qué hacer con ellos.

DOCUMENTACIÓN OFICIAL DEL DÍA

- Tool use, overview: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>
- Como implementar tool use: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/implement-tool-use>

QUÉ CONSTRUIMOS HOY

El cliente de Claude instanciado con tu API key, el schema de la tool `crear_reserva`, y la primera llamada real a la API pasandole esa tool - todavía sin conectar la respuesta a la confirmación final.

```
import os
import anthropic

client = anthropic.Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

tool_reserva = {
    "name": "crear_reserva",
    "description": "Registra una reserva de mesa cuando el cliente da fecha, hora y personas.",
    "input_schema": {
        "type": "object",
        "properties": {
            "fecha": {"type": "string", "description": "Fecha de la reserva"},
            "hora": {"type": "string", "description": "Hora de la reserva"},
            "personas": {"type": "integer", "description": "Cantidad de personas"},
            "nombre": {"type": "string", "description": "Nombre para la reserva"},
        },
        "required": ["fecha", "hora", "personas"],
    },
}

response = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=1024,
    tools=[tool_reserva],
    messages=[{"role": "user", "content": mensaje.texto}],
)
```

- **client = anthropic.Anthropic(api_key=...):** crea el cliente que vamos a reutilizar en cada llamada a Claude de acá en adelante - lee la clave desde la variable de entorno cargada en la preparacion del entorno, nunca escrita directo en el código.
- **name: "crear_reserva":** el nombre que el modelo usa internamente para referirse a esta accion - tiene que ser descriptivo, porque ayuda al modelo a decidir cuando corresponde usarla.
- **description:** la frase que le explica al modelo CUANDO usar esta herramienta. Cuanto más clara y especifica, menos casos raros donde el modelo la use de más o de menos.
- **input_schema de tipo object con properties:** cada propiedad (fecha, hora, personas, nombre) define un dato que el modelo puede llenar, con su tipo (string o integer) y una descripcion propia que ayuda a que lo interprete bien.
- **required: ["fecha", "hora", "personas"]:** marca cuales son obligatorios. nombre queda opcional a proposito, porque no siempre hace falta para tomar la reserva.
- **client.messages.create(model=..., tools=[tool_reserva], messages=[...]):** la llamada real a la API - el parametro tools es lo que habilita al modelo a usar la herramienta, pero es el modelo el que decide solo si corresponde según el mensaje del cliente.

RESULTADO ESPERADO AL CERRAR EL DÍA

Mandarle al modelo "quiero una mesa para 4 el sabado a las 21hs" y ver, en la respuesta cruda de la API, el bloque tool_use con fecha, hora y personas ya extraidos.

Día 4 — 7 de agosto

Leer la respuesta de Claude cuando usa la herramienta, devolver una confirmación real al cliente por WhatsApp, y guardar la reserva en una planilla real que el restaurante pueda ver.

EL CONCEPTO, EN PROFUNDIDAD

La respuesta de la API de Claude viene como una lista de bloques de contenido (content blocks). Cuando el modelo decide usar una herramienta, esa lista incluye un bloque con type: "tool_use", que trae el name de la herramienta y un objeto input con los argumentos ya completados. Tu código tiene que recorrer esos bloques, encontrar el que sea tool_use, y sacar el input de ahí - ese es el dato estructurado real, no texto para parsear a mano.

Con esos datos en la mano, armas el mensaje de confirmación (por ejemplo con un f-string) y ese texto es el que se manda de vuelta por el mismo camino armado el día 2: tu endpoint responde, n8n toma esa respuesta y la manda a la API de WhatsApp con un HTTP Request.

Confirmarle al cliente no alcanza: si la reserva no queda registrada en ningún lado que el restaurante pueda revisar, en los hechos no existe. Por eso hoy el endpoint no solo devuelve el texto de confirmación, sino también los datos de la reserva ya estructurados (fecha, hora, personas, nombre), para que n8n los tome y agregue una fila nueva en una planilla de Google Sheets - ese es el registro real que el restaurante va a mirar, no el chat de WhatsApp.

Aplicado a Accelerate.ai: leer la respuesta estructurada y convertirla en un mensaje humano es el paso que hace que la tecnología se vuelva un producto usable por un cliente real, no solo un experimento técnico. Esta misma lógica (extraer datos, armar una respuesta natural, dejar un registro persistente) es la base del bot productizado (idea 2 del ranking); ahí se reutiliza para distintos rubros con distintas plantillas de confirmación, pero el mecanismo es el mismo que armamos hoy.

PARA COLO

Con los datos que el modelo extrae, armamos una confirmación real y se la mandamos de vuelta al cliente por WhatsApp — como cuando repetís un pedido en voz alta para confirmar que entendiste bien. Pero además, hoy es el día importante para vos: cada reserva confirmada va a quedar anotada en una planilla de Google Sheets que vas a poder abrir vos mismo, sin depender de revisar el WhatsApp del bot — el mismo tipo de planilla que después vas a manejar con Power BI. Desde hoy, si le escribís al bot pidiendo una mesa, te va a contestar confirmando exactamente lo que entendió, y esa reserva va a quedar anotada de verdad.

DOCUMENTACIÓN OFICIAL DEL DÍA

- Como implementar tool use: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/implement-tool-use>

QUÉ CONSTRUIMOS HOY

La función que recorre response.content, extrae el bloque tool_use, arma el texto de confirmación, y devuelve tanto el texto como los datos estructurados de la reserva - listos para que n8n los escriba en Google Sheets antes de mandar la respuesta final al cliente.

```

for bloque in response.content:
    if bloque.type == "tool_use" and bloque.name == "crear_reserva":
        datos = bloque.input
        confirmacion = (
            f"Confirmado: mesa para {datos['personas']} "
            f"el {datos['fecha']} a las {datos['hora']}"
        )
        return {
            "respuesta": confirmacion,
            "reserva": datos, # para que n8n la guarde en Sheets
        }

```

- **for bloque in response.content:** la respuesta de Claude puede traer varios bloques (texto y tool_use mezclados); recorrerlos es la forma estándar de no asumir que siempre viene un solo tipo de contenido.
- **if bloque.type == "tool_use" and bloque.name == "crear_reserva":** filtra específicamente el bloque que nos interesa, ignorando bloques de texto que pueda haber mandado el modelo antes o después.
- **datos = bloque.input:** acá es donde efectivamente sacamos el diccionario ya estructurado - el dato real que veníamos buscando desde el día 3.
- **El f-string de confirmación:** arma el mensaje final combinando los datos extraídos con texto fijo - es exactamente lo que se manda de vuelta al cliente por WhatsApp, usando el camino armado el día 2.
- **"reserva": datos en el return:** además del texto, devolvemos el diccionario completo con fecha, hora, personas y nombre - es lo que n8n necesita para poder escribir una fila en la planilla, no le alcanza con el texto de confirmación.
- **Nuevo nodo Google Sheets en n8n,** insertado entre el HTTP Request que llama a tu FastAPI y el que manda la respuesta por WhatsApp: toma el objeto reserva de la respuesta y agrega una fila nueva a una planilla (fecha, hora, personas, nombre, número de teléfono) - conectada con tu cuenta de Google, apuntando a una planilla creada de antemano con esas columnas.

RESULTADO ESPERADO AL CERRAR EL DÍA

Mandar el mensaje real desde tu WhatsApp y recibir la confirmación con los datos correctos, no un eco genérico - y ver esa misma reserva aparecer como una fila nueva en la planilla de Google Sheets.

Día 5 — 8 de agosto

Manejar el caso en que falta un dato obligatorio, y darle memoria a la conversacion para que no se reinicie en cada mensaje.

EL CONCEPTO, EN PROFUNDIDAD

Cuando un campo está marcado como required en el input_schema pero el cliente no lo menciona, Claude puede devolver una respuesta de texto pidiendo específicamente el dato que falta, en vez de inventarlo - pero solo si tu prompt lo guía a hacer eso (conviene indicarlo explícitamente en el system prompt: "si falta un dato obligatorio, pregúntalo en vez de asumirlo").

El problema más real de un bot de WhatsApp es la memoria: la API de Claude no recuerda nada por sí sola entre llamadas - cada request es independiente. Si el cliente manda "una mesa para el sábado" y, un mensaje después, "somos 4", tu sistema tiene que reconocer que ambos mensajes son de la misma conversación (identificada por el número de teléfono) y mandarle a Claude el historial completo de esa conversación en cada

llamada nueva, no solo el último mensaje. Esto se resuelve guardando, por número de teléfono, la lista de mensajes intercambiados, y agregando el mensaje nuevo a esa lista antes de cada llamada.

Aplicado a Accelerate.ai: la memoria de conversacion no es un detalle menor - sin esto, cualquier bot de WhatsApp de la agencia se rompe apenas un cliente escribe en más de un mensaje, que es la forma más comun en que la gente realmente escribe. Este mismo mecanismo de guardar historial por número de teléfono es el que se reutiliza en el bot productizado para cualquier rubro futuro, no solo para el restaurante.

PARA COLO

Hoy resolvemos dos problemas reales. Primero, qué pasa si al cliente le falta dar un dato obligatorio: el sistema tiene que preguntar específicamente lo que falta, en vez de inventarlo. Segundo, la memoria de la conversación — la IA no recuerda nada entre un mensaje y el siguiente por sí sola, así que si el cliente completa el pedido en dos mensajes separados, nuestro sistema tiene que guardar ese historial (identificado por el número de teléfono) y mandárselo de nuevo a la IA en cada llamada, para que no trate cada mensaje como si fuera de un cliente distinto.

DOCUMENTACIÓN OFICIAL DEL DÍA

- Tool use, overview: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>
- Como implementar tool use: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/implement-tool-use>

QUÉ CONSTRUIMOS HOY

Un diccionario en memoria (conversaciones = {}) que guarda el historial por número de teléfono, y la lógica que arma el mensaje de "falta un dato" cuando corresponde.

```
conversaciones = {} # numero -> lista de mensajes

def agregar_mensaje(numero, rol, texto):
    historial = conversaciones.setdefault(numero, [])
    historial.append({"role": rol, "content": texto})
    return historial
```

- **conversaciones = {}:** un diccionario donde la clave es el número de teléfono y el valor es la lista de mensajes de esa conversacion puntual - así no se mezclan los historiales de distintos clientes.
- **conversaciones.setdefault(número, []):** si es la primera vez que ese número escribe, crea la lista vacia; si ya existe, la reutiliza - evita chequear a mano si el número ya estaba registrado.
- **historial.append({"role": rol, "content": texto}):** agrega el mensaje nuevo con el formato exacto que espera el parametro messages de la API de Claude (rol puede ser "user" o "assistant").
- **Mandar el historial completo** (no solo el último mensaje) en cada llamada a la API es lo que le da al modelo el contexto de todo lo dicho antes, incluyendo datos que el cliente mencionó en mensajes previos.

RESULTADO ESPERADO AL CERRAR EL DÍA

Probar una conversacion real de 2-3 mensajes donde los datos se completan de a poco, y que el bot arme la reserva completa y correcta al final.

Sección 3 — RAG casero para el menú y las FAQ

Día 6 — 9 de agosto

Entender que es un embedding y generar los vectores del FAQ/menú real que escribio Colo.

EL CONCEPTO, EN PROFUNDIDAD

Un embedding es una lista de numeros (un vector) que representa el significado de un texto - textos con significados parecidos quedan matemáticamente "cerca" entre si en ese espacio de numeros, aunque no compartan las mismas palabras. Claude no genera embeddings directamente: Anthropic recomienda usar Voyage AI para esta parte puntual (o, como alternativa, los embeddings de OpenAI) - es normal y esperable combinar proveedores: embeddings de Voyage, generacion de texto con Claude, no hay ningun problema en mezclarlos.

El flujo es: tomas cada fragmento de texto (una pregunta del FAQ, o un plato del menú con su descripcion), se lo mandas a la API de Voyage, y te devuelve un vector. Guardas ese vector junto con el texto original, para más adelante poder compararlo con la pregunta de un cliente.

Plan B si el FAQ de Colo todavía no esta listo para hoy: no hace falta frenar el estudio. Se puede arrancar con 5 o 6 preguntas de ejemplo inventadas (horarios, si tienen wifi, si aceptan tarjeta) para aprender el mecanismo completo, y reemplazarlas por las reales apenas Colo las mande - el código no cambia, solo cambia el contenido de la lista faq.

Aplicado a Accelerate.ai: este mismo mecanismo (convertir contenido de un negocio en embeddings buscables) es el corazon de la idea 3 del ranking (dashboards y reportes) y de cualquier futuro cliente que tenga su propio catalogo, manual o base de preguntas frecuentes. No se reescribe de cero para cada cliente nuevo - se reemplaza el contenido de entrada (el FAQ puntual), pero la lógica de generar y buscar embeddings es siempre la misma.

PARA COLO

Arrancamos con RAG (Retrieval-Augmented Generation, generación de respuestas apoyada en una búsqueda previa) usando el FAQ y el menú que escribiste. El primer paso es convertir cada pregunta y cada plato en un embedding: una representación numérica del significado de ese texto, generada por un servicio especializado aparte (Voyage AI, no el mismo Claude). Dos textos con significado parecido terminan con números parecidos aunque estén redactados distinto — eso es lo que después permite buscar por significado y no por coincidencia exacta de palabras.

DOCUMENTACIÓN OFICIAL DEL DÍA

- Guía de embeddings de Anthropic (Voyage AI): <https://platform.claude.com/docs/en/build-with-claude/embeddings>
- Cookbook oficial de Anthropic para embeddings con Voyage:
https://github.com/anthropics/claude-cookbooks/blob/main/third_party/VoyageAI/how_to_create_embeddings.md

QUÉ CONSTRUIMOS HOY

Un script que toma el FAQ de Colo (una lista de preguntas y respuestas), genera el embedding de cada una con la API de Voyage, y guarda todo en un archivo JSON local (texto + vector).


```
import json
import voyageai

vo = voyageai.Client() # lee VOYAGE_API_KEY del entorno

faq = [
    "Tienen opciones sin TACC? Si, 3 platos aptos para celiacos.",
    "Hasta que hora atienden los sabados? Hasta las 00:30.",
]

resultado = vo.embed(faq, model="voyage-4", input_type="document")
vectores = resultado.embeddings # un vector por fragmento

# Guardamos texto y vector juntos, listos para cargar en cualquier momento
faq_vectores = list(zip(faq, vectores))
with open("faq_embeddings.json", "w") as f:
    json.dump(faq_vectores, f)
```

- **vo = voyageai.Client()**: crea el cliente de Voyage, que automáticamente busca la variable de entorno VOYAGE_API_KEY - conviene tenerla configurada antes de correr el script.
- **faq = [...]**: la lista de textos a convertir - en este caso, cada entrada del FAQ real que escribio Colo.
- **vo.embed(faq, model="voyage-4", input_type="document")**: input_type="document" le indica a Voyage que estos textos son contenido para buscar Después (a diferencia de una pregunta, que usa "query", como se ve el día 7) - Voyage genera vectores levemente distintos según el caso.
- **resultado.embeddings**: la lista de vectores resultante, uno por cada entrada del FAQ, en el mismo orden.
- **list(zip(faq, vectores))** y **json.dump(...)**: junta cada texto con su vector correspondiente y lo guarda en un archivo en disco - sin este paso, los embeddings se perderian apenas termine de correr el script, y el día 7 no tendria de donde leerlos.

RESULTADO ESPERADO AL CERRAR EL DÍA

Correr el script, confirmar que devuelve un vector numerico por cada fragmento del FAQ real, y verificar que el archivo faq_embeddings.json se creo correctamente en el proyecto.

Día 7 — 10 de agosto

Partir el FAQ en fragmentos manejables (chunking) y armar la busqueda por similitud.

EL CONCEPTO, EN PROFUNDIDAD

Cuando un texto es largo, conviene partirlo en fragmentos (chunks) antes de generar los embeddings, porque un embedding de un texto muy largo promedia demasiados temas distintos y pierde precision a la hora de buscar. En un FAQ de restaurante, el chunking natural es por pregunta-respuesta: cada par ya es una unidad de sentido completa.

La busqueda por similitud se hace con similitud coseno: una formula que mide el angulo entre dos vectores - cuanto más chico el angulo, más parecido el significado. Es el producto punto de los dos vectores dividido por el producto de sus magnitudes, y da un número entre -1 y 1. Con numpy, esto son dos o tres lineas de código, no hace falta ninguna base de datos vectorial para un FAQ de este tamaño.

El proceso completo: generas el embedding de la pregunta del cliente (con `input_type="query"` en vez de `"document"` - Voyage optimiza el vector distinto según si es una pregunta o un documento), calculas la similitud contra cada vector guardado el día 6, y te quedas con los 2-3 fragmentos de mayor similitud.

Aplicado a Accelerate.ai: esta función de búsqueda es la pieza que separa un bot que "no entiende nada fuera de lo que programamos a mano" de uno que responde con inteligencia real sobre el contenido del negocio. Es reutilizable tal cual para cualquier cliente futuro de la agencia - lo único que cambia es el contenido del FAQ que se le carga, la lógica de búsqueda queda intacta.

PARA COLO

Con los embeddings generados ayer, hoy armamos la búsqueda real: cuando llega una pregunta de un cliente, la convertimos también en su representación numérica y la comparamos matemáticamente (por similitud coseno, una medida de qué tan parecidos son dos vectores) contra cada entrada del FAQ guardada. Nos quedamos con las 2 o 3 más parecidas, y esas son las que se usan para responder — así el sistema no necesita que la pregunta esté escrita exactamente igual que en el FAQ para reconocerla.

DOCUMENTACIÓN OFICIAL DEL DÍA

- Guía de embeddings de Anthropic (Voyage AI): <https://platform.claude.com/docs/en/build-with-claude/embeddings>
- NumPy, álgebra lineal: <https://numpy.org/doc/stable/reference/routines.linalg.html>

QUÉ CONSTRUIMOS HOY

Cargamos de nuevo los embeddings guardados ayer, y armamos la función `buscar_relevantes(pregunta, top_k=3)` que devuelve los fragmentos del FAQ más relevantes para una pregunta dada.

```
import json
import numpy as np

with open("faq_embeddings.json") as f:
    faq_vectores = json.load(f) # la lista de (texto, vector) del día 6

def similitud_coseno(a, b):
    a, b = np.array(a), np.array(b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def buscar_relevantes(pregunta, faq_vectores, top_k=3):
    emb = vo.embed([pregunta], model="voyage-4", input_type="query").embeddings[0]
    scores = [(similitud_coseno(emb, v), texto) for texto, v in faq_vectores]
    scores.sort(reverse=True)
    return [texto for _, texto in scores[:top_k]]
```

- `json.load(f)` al principio: recupera del disco la lista de (texto, vector) que se guardó el día 6 - sin esto, `faq_vectores` no existiría cada vez que el servidor arranca de nuevo.
- `similitud_coseno(a, b)`: convierte las listas de números a arrays de numpy, y aplica la fórmula (producto punto dividido por el producto de las normas) - devuelve un número entre -1 y 1.

- **buscar_relevantes(pregunta, faq_vectores, top_k=3):** primero convierte la pregunta del cliente en su propio embedding, usando `input_type="query"` esta vez, porque ahora es una pregunta y no contenido a indexar.
- **El list comprehension de scores:** calcula la similitud entre la pregunta y CADA entrada guardada del FAQ, generando una lista de pares (puntaje, texto).
- **scores.sort(reverse=True):** ordena de mayor a menor similitud, para que los más relevantes queden primero.
- **return [texto for _, texto in scores[:top_k]]:** se queda solo con los top_k (acá, 3) textos más relevantes, descartando el puntaje una vez que ya cumplió su función de ordenar.

RESULTADO ESPERADO AL CERRAR EL DÍA

Probar la función con 4-5 preguntas reales del FAQ de Colo (escritas distinto a como estan en el FAQ) y confirmar que siempre trae el fragmento correcto.

Día 8 — 11 de agosto

Unificar todo en el mismo bot - que decida solo si el mensaje es una reserva o una pregunta de FAQ.

EL CONCEPTO, EN PROFUNDIDAD

Hasta ahora hay dos caminos construidos por separado: la extracción de reservas con tool use (días 3-5) y la búsqueda por RAG (días 6-7). Hoy se combinan en un unico flujo de decisión. La forma más simple y robusta: le seguís pasando a Claude la tool `crear_reserva` en cada llamada; si el modelo decide usarla (porque el mensaje es claramente una reserva), segui el camino de los días 3-5. Si el modelo responde con texto normal en vez de `tool_use` (porque interpreto que no es una reserva, sino una pregunta), tratas ese mensaje como una consulta: corres la búsqueda del día 7, agregas los fragmentos encontrados al prompt como contexto, y le pedís a Claude que responda la pregunta usando ese contexto - esta última parte es la esencia de RAG: Retrieval-Augmented Generation, generacion de texto aumentada con información recuperada.

Aplicado a Accelerate.ai: este "switch" entre reserva y pregunta es, en los hechos, el primer paso hacia la idea 4 del ranking (agencia de software): un mismo sistema que decide solo que hacer según el pedido, en vez de que el cliente tenga que elegir un menú de opciones. Cuanto más pulido quede este día, más fácil es después empaquetarlo como producto para otros restaurantes.

PARA COLO

Hoy unificamos los dos caminos que veníamos armando por separado: reservas (con el formulario del día 3) y preguntas de FAQ (con la búsqueda de ayer). El mismo bot, con el mismo mensaje de entrada, decide solo cuál de los dos caminos corresponde — si el modelo detecta una reserva, sigue ese proceso; si no, busca en el FAQ y responde usando esa información como contexto. No hace falta que el cliente elija un menú de opciones ni que nosotros clasifiquemos el mensaje a mano.

DOCUMENTACIÓN OFICIAL DEL DÍA

- Tool use, overview: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>
- Guía de embeddings de Anthropic (Voyage AI): <https://platform.claude.com/docs/en/build-with-claude/embeddings>

QUÉ CONSTRUIMOS HOY

La lógica condicional en el endpoint: si `tool_use` esta presente, camino de reserva; si no, camino de búsqueda más respuesta con contexto.

```
if hay_tool_use(response):
    return manejar_reserva(response)
else:
    fragmentos = buscar_relevantes(mensaje.texto, faq_vectores)
    contexto = "\n".join(fragmentos)
    return responder_con_contexto(mensaje.texto, contexto)
```

- **hay_tool_use(response):** una función chica que revisa si alguno de los bloques de la respuesta es de tipo `tool_use` - decide que camino seguir.
- **Si es True:** `manejar_reserva(response)` reutiliza exactamente la lógica de los días 3 y 4, sin duplicar código.
- **Si es False:** se asume que es una pregunta. `buscar_relevantes(...)` trae los fragmentos del FAQ (día 7), y se unen con `"\n".join(fragmentos)` para armar un bloque de texto de contexto.
- **responder_con_contexto(...):** manda ese contexto junto con la pregunta original a Claude, pidiéndole que responda basandose en lo que se le paso, no de memoria propia - reduce muchísimo el riesgo de que invente información sobre el restaurante.

RESULTADO ESPERADO AL CERRAR EL DÍA

En una misma conversacion de WhatsApp, alternar una pregunta de menú y un pedido de reserva, y que el bot responda bien a las dos sin mezclarlas.

Sección 4 — Pulido y testing de punta a punta

Día 9 — 12 de agosto

Que el bot nunca se quede mudo - manejar errores y casos fuera de lo esperado.

EL CONCEPTO, EN PROFUNDIDAD

Hay tres tipos de fallas a contemplar: que la llamada a la API de Claude falle (por red, limite de uso, o timeout), que el cliente escriba algo totalmente fuera de tema, y que la búsqueda de RAG no encuentre nada suficientemente relevante. En los tres casos, la respuesta correcta nunca es no responder nada - un webhook que no devuelve nada dentro de un tiempo razonable hace que WhatsApp lo marque como fallido, y un bot mudo genera más desconfianza que uno que a veces se equivoca. La práctica estándar es envolver las llamadas externas en un bloque `try/except`, y tener siempre un mensaje de resguardo (fallback) del estilo "no pude entender bien eso, podés reformularlo?" o, para preguntas sin buen match de RAG, "no tengo esa información a mano, te va a responder alguien del local".

Aplicado a Accelerate.ai: para un cliente real que esta pagando una mensualidad (como planeamos cobrar en el modelo de precios de la agencia), un bot que se cae en silencio es directamente un problema de servicio, no un detalle técnico menor. Este manejo de errores es lo que hace la diferencia entre un prototipo y algo que se puede vender con confianza a cualquier negocio, no solo al restaurante de Colo.

PARA COLO

Un sistema real tiene que anticipar que algo va a fallar en algún momento: que la IA tarde en responder, que el cliente escriba algo totalmente fuera de tema, o que no encontremos nada relevante para una pregunta. Hoy nos aseguramos de que, en cualquiera de esos casos, el bot siempre devuelva algo coherente — aunque sea pedir que reformulen — en vez de quedarse sin responder. Un sistema que falla en silencio genera mucha más desconfianza que uno que a veces pide una aclaración.

DOCUMENTACIÓN OFICIAL DEL DÍA

- FastAPI, manejo de excepciones: <https://fastapi.tiangolo.com/tutorial/handling-errors/>
- Tool use, overview: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>

QUÉ CONSTRUIMOS HOY

Manejo de excepciones alrededor de las llamadas a Claude y a Voyage, y los mensajes de resguardo para los tres casos de falla.

```
try:
    response = client.messages.create(...)
except Exception:
    return {"respuesta": "No pude entender eso, me lo repetis de otra forma?"}
```

- **try/except alrededor de la llamada a Claude:** si la API falla por cualquier motivo (timeout, límite de uso, error de red), el except captura el problema en vez de que el programa se caiga entero.
- **El mensaje de fallback dentro del except:** siempre devuelve algo coherente al cliente, aunque sea pedirle que reformule - nunca se deja al webhook sin responder nada.
- **El mismo patron se replica** alrededor de la llamada a Voyage (embeddings), por si ese servicio falla independientemente de Claude.

RESULTADO ESPERADO AL CERRAR EL DÍA

Mandar mensajes raros a proposito (vacíos, en otro idioma, sin sentido) y confirmar que el bot siempre responde algo razonable.

Día 10 — 13 de agosto

Testear todo el sistema con los guiones de conversacion reales que escribio Colo.

EL CONCEPTO, EN PROFUNDIDAD

Esto es testing de aceptacion: probar el sistema con casos reales, escritos por alguien que no programo el sistema. Es distinto (y más valioso en este punto) que seguir probando con los mensajes que vos mismo imaginaste mientras programabas, porque quien construye algo tiende a probarlo de la forma en que espera que funcione, no de la forma en que un cliente real lo va a usar. Corre cada uno de los guiones de Colo contra el bot ya completo, y anota exactamente donde falla: no extrae bien un dato?, no encuentra la pregunta correcta en el

FAQ?, se cuelga con algun formato de mensaje particular?

Aplicado a Accelerate.ai: este es el paso que en cualquier desarrollo de software separa "funciona en mi máquina" de "funciona de verdad" - y es el mismo proceso, más corto, que se va a repetir cada vez que se arme una automatización para un cliente nuevo de la agencia.

PARA COLO

Hoy es tu día. Todo lo que construimos se prueba contra los guiones de conversación reales que escribiste — mensajes tal como los escribe un cliente de verdad, no como los imagina quien programó el sistema. Es una etapa de testing real, y tu trabajo es identificar exactamente dónde el bot responde mal o no entiende algo que para vos, con tu experiencia en el salón, resulta obvio.

QUÉ CONSTRUIMOS HOY

No se programa nada nuevo hoy. El trabajo es ejecutar, uno por uno, los 15-20 guiones que escribió Colo contra el sistema completo (FastAPI, Claude, RAG, n8n y WhatsApp), anotando con precisión qué mensaje se mandó, qué se esperaba, y qué contestó el bot realmente.

- Conviene armar una lista simple (una tabla o un documento compartido) con tres columnas: mensaje mandado, respuesta esperada, respuesta real del bot.
- Priorizar los casos que rompen el flujo principal (no puede tomar una reserva básica) por sobre los detalles menores (una respuesta un poco torpe en un caso raro).
- No corregir nada todavía - el objetivo de hoy es solo relevar y documentar las fallas, para poder atacarlas con criterio mañana.

RESULTADO ESPERADO AL CERRAR EL DÍA

Una lista escrita de que fallo, lista para arreglar mañana.

Día 11 — 14 de agosto

Corregir lo que fallo el día 10 y cerrar la ventana de preparacion con el sistema funcionando de punta a punta.

EL CONCEPTO, EN PROFUNDIDAD

No hay concepto nuevo - es el día de aplicar lo aprendido en los 10 días anteriores para resolver los problemas concretos que aparecieron en el testing. Es normal (y esperable) que la lista del día 10 no este vacia; ningun sistema pasa el testing real a la primera. Priorizar lo que rompe el flujo principal (no puede tomar una reserva básica) antes que los detalles menores (una respuesta un poco torpe en un caso raro).

Aplicado a Accelerate.ai: cerrar bien esta ventana de preparacion es lo que permite que, a partir del 14 de agosto, el foco pase de "aprender la base técnica" a "construir con esa base ya andando" - sin arrastrar deuda técnica del arranque a los proximos clientes de la agencia.

PARA COLO

Hoy corregimos lo que encontramos ayer. Es normal que la primera ronda de testing real muestre varios problemas — ningún sistema pasa a la primera. Al final del día volvemos a correr los mismos casos para confirmar que ahora sí funcionan, y ese es el cierre real de estos 11 días antes de que arranque tu curso de Excel.

QUÉ CONSTRUIMOS HOY

Se toma la lista de fallas del día 10, ordenada de más grave a más leve, y se corrige una por una: primero lo que impide completar una reserva o pedido básico, después los casos raros o las respuestas poco naturales.

- Cada corrección se vuelve a probar de inmediato con el mismo mensaje que la había hecho fallar, antes de pasar a la siguiente — no se acumulan arreglos sin verificar.
- Si una falla resulta más grande de lo esperado (por ejemplo, un problema de diseño en cómo se separan reservas y preguntas), se anota aparte para no perder el resto del día en un solo caso.
- Al final del día, se corre la batería completa de guiones de Colo una vez más, de punta a punta, no solo los que habían fallado — para confirmar que las correcciones no rompieron algo que antes funcionaba.

RESULTADO ESPERADO AL CERRAR EL DÍA

Correr de nuevo todos los guiones de Colo y que todos pasen - ese es el cierre real de la ventana de preparacion, justo antes de que Colo arranque su curso de Excel.

Glosario

API

El mecanismo estandar por el que dos programas distintos se pasan informacion - en esta guia, la forma en que WhatsApp y nuestro servidor se comunican.

Endpoint

Una URL puntual de una API que responde a un tipo de peticion concreto - en esta guia, /webhook.

Webhook

Un endpoint pensado para que OTRO sistema (Meta, en este caso) le avise al nuestro cuando pasa algo, en vez de que nosotros le preguntemos todo el tiempo.

Embedding

Una representacion numerica (un vector) del significado de un texto, generada por un modelo especializado - permite comparar textos por significado, no por coincidencia exacta de palabras.

Tool use / function calling

La tecnica por la que un modelo de IA devuelve datos estructurados (segun un esquema que vos definis) en vez de texto libre, dejando la ejecucion real en manos de tu codigo.

RAG (Retrieval-Augmented Generation)

Generacion de texto apoyada en una busqueda previa: se recupera informacion relevante de una base propia y se la pasa al modelo como contexto antes de que responda.

Chunking

Partir un texto largo en fragmentos mas chicos antes de generar embeddings, para no perder precision en la busqueda.

Similitud coseno

Una formula que mide que tan parecidos son dos vectores, calculando el angulo entre ellos - da un numero entre -1 y 1.

Async / await

Formas de escribir codigo en Python que permiten atender otra peticion mientras se espera algo lento (como una llamada de red), en vez de quedar bloqueado.

Pydantic / BaseModel

La libreria que usa FastAPI para validar automaticamente la forma de los datos que entran y salen de un endpoint.

Sub-cuenta (GoHighLevel)

Un espacio independiente dentro de la plataforma, uno por cliente, con su propia marca y facturacion - la pieza que permite escalar de un cliente a varios sin mezclar sus datos.

Ventana de 24 horas (WhatsApp)

El plazo dentro del cual se puede mandar texto libre en respuesta a un cliente - fuera de esa ventana, Meta exige usar una plantilla de mensaje pre-aprobada.